MIRAGE ENGINE V6

Unified Computational Runtime Architecture Specification

Differential Continuation-Oriented Structural Computing Substrate

---

Abstract

Mirage Engine V6 represents the transition of Mirage from an experimental computational runtime into a fully deterministic structural-computation operating substrate.

Mirage V6 is not a traditional game engine. It is not a renderer with simulation attached. It is not an ECS framework. It is not a task graph runtime. It is not an actor system. It is not a scene-graph architecture.

Mirage V6 instead defines:

- a differential computational field runtime
- a continuation-oriented execution substrate
- a deterministic topology realization system
- a structural continuity propagation layer
- a replay-safe morphogenic runtime
- a sparse computational operating architecture
- a residency-governed execution environment
- a topology-aware propagation kernel
- a continuation-native scheduling substrate
- a deterministic structural persistence system

Mirage V6 unifies:

- V3 differential propagation concepts
- V4 continuation-oriented sparse execution
- V5 morphogenic structural runtime
- V6 deterministic structural governance and realization contracts

into a single computational architecture.

Unlike conventional engines that organize execution around objects, entities, actors, scenes, or frame graphs, Mirage organizes execution around:

- computational pressure
- propagation frontiers
- deterministic continuities
- realization sequences

- topology continuity
- sparse activation domains
- continuation authority
- morphogenic persistence
- runtime structural stabilization

Mirage V6 treats execution itself as a deterministic evolving computational field.

PART I — PHILOSOPHICAL FOUNDATION

1. Mirage Is Not A Traditional Engine

Traditional engines fundamentally organize around ownership hierarchies.

Examples:

Engine	Central Authority
Unreal Engine 5	Actor Hierarchy
Unity DOTS	ECS Chunks
Frostbite	Task Graph
Godot	Scene Tree
O3DE	Component Graph
Bevy	ECS Scheduler

Mirage V6 rejects all of these assumptions.

Mirage instead defines:

Mirage V6 Concept	Architectural Authority
Differential Field	Computational Truth
Continuation	Execution Unit
Frontier	Change Boundary
OASIS	Residency Authority
CEK Runtime	Execution Semantics
MKR	Sparse Simulation Kernel

Mirage V6 Concept	Architectural Authority
MTS	Topology Authority
Morphogenic Runtime	Structural Persistence

Mirage therefore behaves more like:

- a computational operating substrate
- a sparse execution operating system
- a deterministic structural runtime

than a traditional engine.

---

2. The Core Mirage Principle

Mirage is built on a foundational assumption:

> Most computation is dormant most of the time.

Traditional engines repeatedly recompute large execution spaces every frame.

Traditional execution:

```
for every frame:
    iterate all entities
    run all systems
    update all visible objects
```

Mirage execution:

```
detect differential changes
build propagation frontier
expand deterministic halo
activate sparse regions
emit continuations
realize execution
stabilize continuity
persist structure
```

This leads to:

- sparse execution

- differential recomputation
- frontier-local scheduling
- deterministic propagation
- locality-oriented execution
- continuation-native simulation
- replay-safe runtime behavior

PART II — EVOLUTION OF MIRAGE

3. Mirage V1

Mirage V1 explored:

- GPU-driven rendering
- SIMD execution
- runtime experimentation
- low-level abstraction systems

Limitations:

- execution remained frame-centric
- no sparse propagation
- no continuation substrate
- no topology semantics

---

4. Mirage V2

Mirage V2 introduced:

- data-oriented runtime structures
- streaming experiments
- simulation fields
- execution graph prototypes

Limitations:

- recomputation still global
- scheduling still heuristic
- no deterministic structural runtime
- no continuation isolation

---

5. Mirage V3

Mirage V3 introduced:

- activation fields
- frontier propagation
- sparse validation runtime
- differential scheduling preparation
- topology-aware propagation
- emission systems
- execution bridges

V3 was the first version where Mirage stopped behaving like a conventional engine.

However V3 still contained transitional architecture:

- renderer-owned authority leakage
- incomplete continuation runtime
- topology mixed with mathematical substrate
- compatibility scheduling remnants
- partial sparse authority

---

6. Mirage V4

Mirage V4 introduced:

- MKR runtime
- OASIS residency authority
- CEK continuation runtime
- sparse computational domains
- continuation-oriented scheduling
- deterministic sparse propagation
- topology-native activation systems

Mirage V4 transformed Mirage into:

> a sparse computational operating substrate.

But topology and structural continuity were still transitional.

---

7. Mirage V5

Mirage V5 introduced the first true structural runtime layer.

Major additions:

- topology extraction
- topology authority isolation
- morphogenic runtime substrate
- structural continuity systems
- deterministic propagation sequences
- replay-safe realization frames
- continuity lifecycle management
- reinforcement persistence
- structural replay governance

V5 transformed Mirage from:

```
sparse continuation runtime
```

into:

```
deterministic structural computational substrate
```

---

8. Mirage V6

Mirage V6 represents:

- full deterministic governance
- structural realization sealing
- continuity replay equivalence
- topology realization contracts
- morphogenic persistence stabilization
- authority-bound execution governance
- canonical structural replay systems
- deterministic runtime verification

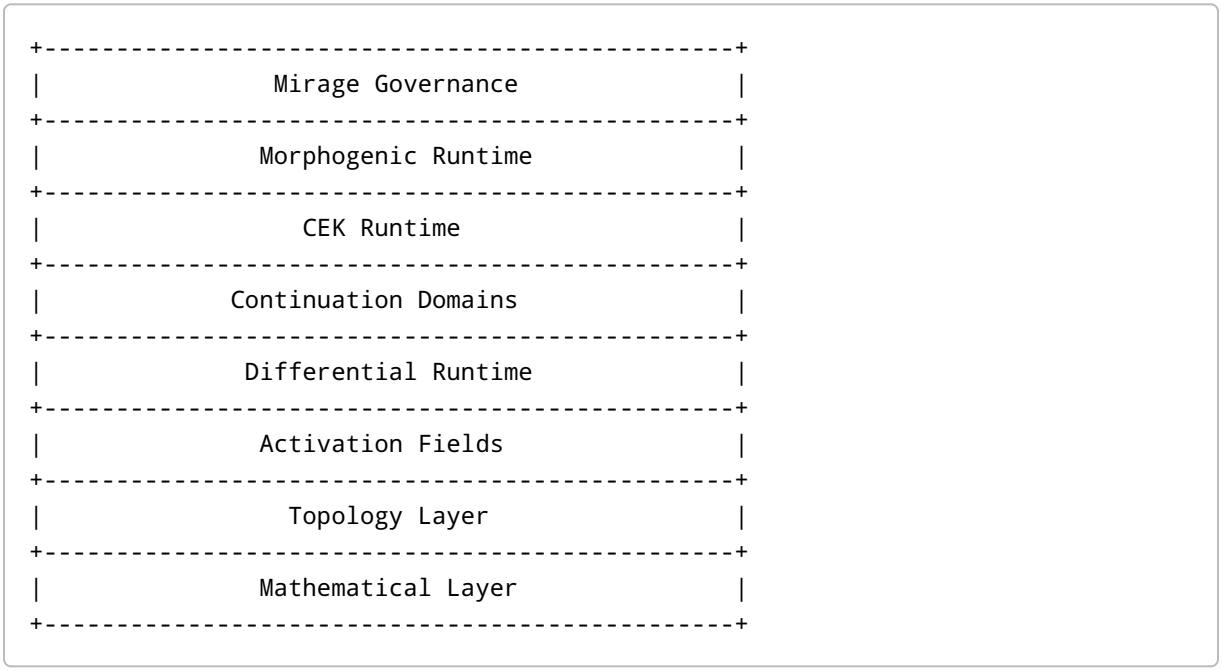
Mirage V6 fully separates:

Layer	Responsibility
mirage-math	Numerical primitives

Layer	Responsibility
mirage-matrix	Mathematical substrate
mirage-mts	Topology authority
mirage-morphogenic	Structural continuity
mirage-cek	Continuation semantics
mirage-executor	Passive execution
mirage-mkr-core	Runtime orchestration
OASIS	Residency authority

PART III — CORE ARCHITECTURE

9. Mirage Core Runtime Layers



Each layer has isolated authority.

No layer may leak ownership.

---

10. Runtime Authority Hierarchy

```
Governance Layer
    ↓
Continuation Authority
    ↓
MKR Runtime
    ↓
Topology Runtime
    ↓
Morphogenic Runtime
    ↓
Residency Runtime
    ↓
Passive Execution
```

This prevents:

- hidden scheduling ownership
- renderer authority inversion
- topology mutation leakage
- execution side effects
- nondeterministic propagation

PART IV — THE MKR RUNTIME

11. MKR — Mirage Kernel Runtime

MKR is the computational heart of Mirage.

Responsibilities:

- differential propagation
- frontier construction
- sparse scheduling
- activation evolution
- continuation emission
- region activation
- structural influence propagation

MKR is NOT:

- a renderer
- an ECS
- a physics engine
- a task graph
- an actor runtime

MKR is effectively:

> a sparse computational operating kernel.

---

12. Activation Fields

The activation field represents computational excitation.

Example:

```rust
pub struct ActivationCell {
    pub activation: f32,
    pub pressure: f32,
    pub execution_probability: f32,
    pub entropy: f32,
}
```

Activation cells are NOT gameplay objects.

They represent:

- computational potential
- propagation energy
- continuation likelihood
- structural pressure

This is fundamentally different from ECS entities.

---

13. Differential Propagation

Traditional engines recompute everything.

Mirage recomputes only changed regions.

Mirage propagation pipeline:

```
changed cells
    ↓
frontier generation
    ↓
halo expansion
    ↓
sparse propagation
    ↓
continuation emission
    ↓
realization
```

Benefits:

- reduced memory bandwidth
- cache locality
- sparse execution
- replay-safe scheduling
- minimized synchronization

---

14. Frontier Runtime

The frontier represents the boundary between:

- stable structure
- changing structure

This is one of the most important concepts in Mirage.

The frontier determines:

- execution activation
- propagation scope
- continuation wakeup
- structural stabilization
- residency prediction

PART V — TOPOLOGY SUBSTRATE

15. Mirage Topology Substrate (MTS)

MTS fully owns runtime topology.

MTS responsibilities:

- topology graphs
- lane relationships
- propagation adjacency
- deterministic traversal
- influence propagation
- topology continuity

MTS must remain:

- deterministic
- replay-safe
- traversal-stable
- allocator-independent

MTS must NEVER own:

- execution authority
- continuation realization
- structural persistence
- AI adaptation

---

16. Deterministic Traversal

Mirage V6 forbids:

- hash-order iteration
- unstable graph traversal
- allocator-dependent ordering
- nondeterministic propagation

Mirage instead uses:

- indexed traversal
- stable sorting
- lexicographic ordering
- deterministic sequence indices

Example:

```
edges.sort_unstable_by(|a, b| a.cmp(b));
```

This guarantees replay equivalence.

PART VI — MORPHOGENIC RUNTIME

17. Structural Runtime

The morphogenic runtime is one of Mirage's most important innovations.

Traditional engines have no equivalent.

The morphogenic layer handles:

- structural continuity
- persistence stabilization
- continuity replay
- deterministic structural decay
- structural reinforcement memory
- realization history
- continuity diffs
- replay-safe snapshots

This is NOT AI.

This is NOT autonomous adaptation.

It is:

> deterministic structural evolution infrastructure.

---

18. Structural Continuity

Continuity represents persistent computational structural state across ticks.

Example:

```rust
pub struct StructuralContinuityField {
    continuity: Vec<f32>,
}
```

Continuity fields preserve:

- structural persistence
- deterministic replay
- continuity stabilization
- propagation memory

---

19. Reinforcement Memory

Mirage V6 introduces reinforcement persistence.

Traditional engines discard most transient propagation information.

Mirage preserves:

- reinforcement accumulation
- continuity stabilization
- propagation persistence
- deterministic decay history

This enables:

- stable structural replay
- continuity equivalence
- deterministic evolution

---

20. Structural Replay

Mirage supports replay-safe realization.

Replay pipeline:

```
topology propagation
    ↓
structural descriptors
    ↓
realization sequence
```

```
    ↓
continuity snapshots
    ↓
replay validation
```

Replay equivalence guarantees:

- same inputs
- same propagation
- same snapshots
- same realization order
- same outputs

across executions.

PART VII — CEK CONTINUATION RUNTIME

21. CEK Runtime

Mirage uses a continuation-oriented execution model.

CEK stands for:

- Control
- Environment
- Kontinuation

Traditional engines execute:

```
Update()
Tick()
Task()
```

Mirage executes:

```
continuations
```

A continuation represents:

- suspended computation
- resumable execution
- future computational intent

- sparse execution authority

---

22. Continuation Lifecycle

```
Dormant
    ↓
Predicted
    ↓
Activated
    ↓
Executing
    ↓
Suspended
    ↓
Resumed
    ↓
Completed
```

Continuations are:

- sparse
- locality-aware
- replay-safe
- migration-capable

PART VIII — OASIS RESIDENCY SYSTEM

23. OASIS

OASIS is the residency authority layer.

Traditional engines incorrectly let renderers own residency.

Mirage rejects this.

Mirage authority:

```
MKR determines importance
    ↓
OASIS determines residency
```

```
    ↓
Renderer performs passive upload
```

The renderer NEVER owns streaming policy.

---

24. Continuation-Aware Residency

Traditional engines stream assets based on visibility.

Mirage streams based on:

- continuation prediction
- propagation pressure
- activation probability
- structural continuity

This creates:

- predictive residency
- reduced latency
- continuation locality
- sparse memory efficiency

PART IX — RENDERING ARCHITECTURE

25. Mirage Renderer

Mirage rendering is differential.

Traditional rendering:

```
update scene graph
rewrite visibility buffers
recompute frame graph
```

Mirage rendering:

```
apply sparse cell changes
update differential buffers
commit frontier-local updates
```

The renderer is passive.

It does not own:

- scheduling
- residency
- simulation
- topology

PART X — EXECUTION MODEL

26. Mirage Executor

Mirage execution flow:

```
Frontier
    ↓
Emission Gate
    ↓
ExecutionRequest
    ↓
Continuation Domain
    ↓
CEK Runtime
    ↓
Passive Executor
```

This guarantees:

- deterministic wakeup
- locality-aware execution
- sparse scheduling
- replay-safe continuation execution

PART XI — GOVERNANCE & VALIDATION

27. Deterministic Governance

Mirage V6 introduces runtime governance.

Governance responsibilities:

- invariant enforcement
- replay validation
- authority boundary validation
- structural equivalence verification
- continuation sequencing validation

---

28. Validation Runtime

Mirage runs:

```
authoritative runtime
    +
validation runtime
```

Outputs are compared.

Mirage tracks:

- drift
- divergence
- replay mismatch
- frontier instability
- propagation inconsistency

This creates mathematically verifiable correctness.

PART XII — MIRAGE SUBSYSTEMS

29. Major Mirage Crates

mirage-core

Core contracts, governance, invariants, validation.

mirage-math

SIMD and differential mathematical substrate.

mirage-matrix

Low-level mathematical structures.

mirage-mts

Topology authority and traversal.

mirage-morphogenic

Structural continuity and replay substrate.

mirage-cek

Continuation realization semantics.

mirage-executor

Passive execution backend.

mirage-renderer

Differential rendering backend.

mirage-platform

Hardware and profiling abstraction.

mirage-query

Columnar and kernel query runtime.

mirage-memory-oasis

Residency and memory orchestration.

PART XIII — MIRAGE VS MODERN SYSTEMS

30. Mirage vs Unreal Engine 5

Topic	Unreal Engine 5	Mirage V6
Authority	Actor Hierarchy	Differential Runtime
Scheduling	Task Graph	Continuation Runtime
Streaming	Visibility-based	Continuation-based
Simulation	Object-centric	Computational-field-centric
Execution	Frame-oriented	Sparse differential
Structural Replay	Limited	Deterministic

---

31. Mirage vs Unity DOTS

Topic	Unity DOTS	Mirage V6
Execution Unit	ECS Chunk	Continuation
Runtime Model	Batch Iteration	Sparse Propagation
Streaming	ECS Subscene	OASIS
Topology	Minimal	Native Runtime Authority
Replay Equivalence	Partial	Structural Determinism

---

32. Mirage vs Frostbite

Topic	Frostbite	Mirage V6
Scheduling	Task Graph	Continuation Runtime
Streaming	Sector-based	Continuation-aware
Execution	Job-driven	Frontier-driven
Structural Persistence	Limited	Native Runtime Layer

PART XIV — V6 DETERMINISTIC INVARIANTS

33. Core Runtime Invariants

Mirage V6 guarantees:

- stable traversal ordering
- replay equivalence
- deterministic realization
- topology authority isolation
- passive execution ownership
- continuation determinism
- sparse/full parity
- continuity reproducibility

Forbidden behaviors:

- hidden authority
- nondeterministic traversal
- allocator-order execution
- topology mutation leakage
- renderer residency ownership
- speculative adaptation
- autonomous mutation

PART XV — FUTURE OF MIRAGE

34. Mirage V7 Directions

Possible future systems:

- distributed continuation domains
- GPU continuation realization
- NUMA-aware propagation domains
- structural diffusion compression
- multi-node sparse propagation
- continuity streaming fabrics

---

35. Final Conclusion

Mirage Engine V6 represents a radical departure from traditional engine architecture.

It replaces:

- objects
- actors
- scenes
- ECS scheduling
- task graphs
- renderer-owned authority
- full-frame recomputation

with:

- differential execution
- continuation-oriented realization
- topology-native propagation
- structural continuity systems
- deterministic replay
- sparse computational authority
- morphogenic persistence
- governance-driven runtime validation

Mirage V6 is therefore not merely a game engine.

It is:

> a deterministic structural computational operating substrate.

Mirage combines:

- sparse systems theory
- continuation machines
- topology runtime research
- deterministic replay systems
- structural propagation
- differential simulation
- residency orchestration
- continuation-oriented execution

into a unified runtime architecture.

Mirage V6 ultimately attempts to redefine real-time computation itself by treating execution as a sparse evolving structural field rather than a static frame loop.